# Advanced Communication, Performance and Security

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Universidade de Aveiro

# Introduction: The Reality of Networks

# Beyond the “Plug and Play” I

In earlier sessions, we treated the network as a simple wire connecting two points.

- In a laboratory or home environment, this often works flawlessly.
- Configuration is minimal, and the speed is usually high.
- This is the “ideal” scenario that rarely exists in professional environments.

# Beyond the “Plug and Play” II

Real-world networks are complex, heterogeneous, and often hostile.

- **Complexity:** Data travels through dozens of routers and switches.
- **Control:** Organizations (like universities) impose strict rules on traffic.
- **Limitations:** Not every port is open; not every protocol is allowed.
- **Objective:** Learn to navigate, measure, and bridge these complex systems.

# The “eduroam” Example I

Consider the **eduroam** network used at the University of Aveiro.

- It is a global roaming service for research and education.
- It is highly secure, using WPA2-Enterprise for authentication.
- However, from a networking perspective, it is a **closed** environment.

# The “eduroam” Example II

Why is eduroam “closed” or “restricted”?

- **Security:** To prevent malware from spreading between thousands of students.
- **Resource Management:** To ensure one user doesn’t consume all the bandwidth.
- **Port Filtering:** Often, only common ports like 80 (HTTP) and 443 (HTTPS) are open.
- **The Problem:** How do you run a custom database or a game server in such a restricted network?

# Part I: Network Performance Analysis

# Why Measure Performance? I

"The network is slow" is the most common complaint in IT.

- To a student, it means Netflix is buffering.
- To a company, it means losing thousands of euros per minute.
- To an engineer, it's a **metric** that must be quantified.

# Why Measure Performance? II

We measure performance to:

- **Validate Infrastructure:** Does the hardware meet the specifications?
- **Troubleshoot:** Is the problem in the local Wi-Fi or the global ISP?
- **Planning:** When do we need to upgrade our links?
- **Benchmarks:** Compare different protocols (e.g., TCP vs UDP).

# Key Performance Metrics: Bandwidth

**Bandwidth** is often misunderstood as “speed.”

- It represents the **maximum capacity** of the communication channel.
- Analogy: The number of lanes on a highway.
- Unit: bits per second (bps, Mbps, Gbps).
- Having 1Gbps bandwidth doesn’t guarantee your file transfer will be that fast.

# Key Performance Metrics: Throughput

**Throughput** is the actual amount of data successfully delivered.

- It is the “real-world” speed you experience.
- It is always less than or equal to the bandwidth.
- Influenced by: Protocol overhead, errors, and congestion.
- Analogy: The actual number of cars passing a point per second.

# Key Performance Metrics: Latency

**Latency** is the time delay between a request and a response.

- Often called "ping" or RTT (Round Trip Time).
- Critical for real-time applications (Gaming, VoIP, Video Calls).
- High latency makes a fast link (high bandwidth) feel "sluggish."
- Analogy: The time it takes for a car to drive from Aveiro to Lisbon.

# Key Performance Metrics: Jitter

**Jitter** is the variation in latency over time.

- If one packet takes 20ms and the next takes 100ms, you have high jitter.
- It causes "stuttering" in video and audio streams.
- Buffering is the common solution to mask jitter.

# Measuring Capacity: `iperf3` I

`iperf3` is the tool used to determine the "true" limit of a link.

- It eliminates variables like disk speed or browser processing.
- It purely tests the network stack.
- Requires two points: a **Server** and a **Client**.

## Measuring Capacity: `iperf3` II

Why not just use a web “Speedtest”?

- **Control:** `iperf3` lets you choose the protocol (TCP, UDP, SCTP).
- **Direction:** You can test upload and download separately or simultaneously.
- **Duration:** You can run tests for hours to find intermittent drops.
- **Precision:** It gives raw technical data, not a simplified “bar.”

# TCP vs. UDP Testing with `iperf3`

- **TCP Test:** Measures how well the network handles reliable, ordered data.
  - `iperf3 -c <ip>`
- **UDP Test:** Measures raw throughput and packet loss.
  - `iperf3 -c <ip> -u -b 100M`
  - Important: UDP does not slow down when the network is full; it just drops packets.

# Part II: Network Diagnostics

# The Diagnostic Mindset

When communication fails, we follow the path of the packet.

- We start at the **Local Host** (Interface/IP).
- We move to the **Local Gateway** (Router).
- We cross the **ISP Network**.
- We reach the **Remote Destination**.

# Diagnostic Tool: `ping`

The most basic, yet essential, tool.

- Uses **ICMP** (Internet Control Message Protocol).
- "Are you there?" -> "Yes, I am."
- Tells us two things: **Reachability** and **Latency**.
- **Warning:** On networks like eduroam, many servers block `ping` for security.

# Diagnostic Tool: `traceroute` I

When `ping` works but the service is slow, we need to see the path.

- It shows every router (“hop”) between you and the destination.
- How it works: It sends packets with a low “Time to Live” (TTL).
- Every router reduces the TTL. When it hits 0, the router sends an error back.
- These errors tell us the router’s identity.

# Diagnostic Tool: `traceroute` II

- **Limitation:** Standard `traceroute` is a static snapshot.
- It only shows the path for those specific packets at that moment.
- In dynamic networks, paths can change constantly.

# Advanced Diagnostics: `mtr` I

`mtr` (My Traceroute) is `ping` and `traceroute` combined on steroids.

- It doesn't run once; it probes the path **continuously**.
- It builds a live table of statistics for every router on the path.
- It shows the **Packet Loss %** at every stage.

# Advanced Diagnostics: `mtr` II

- **Justification:** If you see 0% loss at hop 1 and 2, but 50% loss at hop 3, you know exactly where the bottleneck is.
- Essential for reporting issues to network administrators.
- Command: `mtr google.com`

# Packet Sniffing: `tcpdump` I

When diagnostics say the “pipe” is fine, but the application fails.

- We need to inspect the **content** of the communication.
- `tcpdump` is a command-line packet analyzer.
- It captures packets directly from the network interface.
- It allows us to see if the “handshake” is happening or if data is corrupted.

# Packet Sniffing: `tcpdump` II

Why use `tcpdump` instead of the graphical Wireshark?

- **Remote Access:** Most servers do not have a graphical interface.
- **Efficiency:** `tcpdump` uses very little memory and CPU.
- **Automation:** You can script `tcpdump` to start when a specific event occurs.
- **Privacy:** You can filter to only see the headers (metadata) without the payload.

# Part III: Communication Efficiency

# The Cost of Communication

Data transfer is not free.

- **Time:** Large backups can take hours or days.
- **Money:** Cloud providers charge for data “egress” (leaving the network).
- **Power:** Moving data across the globe consumes significant electricity.
- **Reliability:** The longer a transfer takes, the higher the chance of a failure.

# Improving the Model: `rsync` I

The traditional model (SCP/FTP) is "Copy Everything."

- If you have a 1GB file and change 1 line, you send 1GB again.
- This is extremely inefficient.
- **The `rsync` Model:** "Only Copy the Differences."

# Improving the Model: `rsync` II

How does `rsync` know what changed?

- It uses a **checksum** algorithm to compare blocks of files.
- Only the blocks that are different are transmitted.
- It compresses data “on the fly” before sending it.
- It can preserve all file metadata (permissions, owners, times).

# `rsync` in the Real World

- **Backups:** Keeping a local folder identical to a remote server.
- **Website Deployment:** Pushing only the updated HTML/CSS files to the server.
- **Resuming:** If the connection drops at 90%, `rsync` restarts from where it left off.
- Command: `rsync -avz --progress ./local/ user@remote:/data/`

# Part IV: Secure Tunnels and Bridges

# The Firewall Problem

In networks like **eduroam**, you are often trapped behind a strict firewall.

- You want to access your home PC (port 22). **Blocked.**
- You want to access a private database (port 5432). **Blocked.**
- Only “Standard” web traffic (80/443) is allowed.

# The Tunnel Solution

A **Tunnel** is a way to wrap a forbidden protocol inside an allowed one.

- We use **SSH** (Secure Shell) as the “wrapper.”
- Since SSH traffic is encrypted, the firewall cannot see what is inside.
- It sees “SSH traffic” and allows it. Inside, we can be running anything.

# Local Port Forwarding (`-L`) I

"Bring the remote service to me."

- You are at the University (eduroam).
- You need to access a server at home that is not public.
- You "tunnel" the remote port to your local machine.

# Local Port Forwarding (`-L`) II

- **Example:** `ssh -L 8080:localhost:5432 user@home-server`
- Now, you open your local browser to `localhost:8080`.
- The traffic goes through the SSH tunnel and hits the database at home.
- To the University network, you are just "using SSH."

# Remote Port Forwarding (`-R`) I

"Expose my local machine to the world."

- You are developing a website on your laptop.
- You want a friend to see it, but you have no public IP.
- You tunnel your local port to a public server.

# Remote Port Forwarding (`-R`) II

- **Example:** `ssh -R 8080:localhost:80 user@public-server`
- Your friend goes to `http://public-server:8080`.
- The request travels down the tunnel to your laptop.
- This bypasses the NAT/Firewall of the network you are in.

# Dynamic Port Forwarding (`-D`) I

“Use the remote server as my eyes.” (The SOCKS Proxy).

- In some networks, certain websites might be blocked.
- Or you want to browse the web as if you were in another country.
- A Dynamic Tunnel turns your SSH connection into a **Proxy**.

# Dynamic Port Forwarding (`-D`) II

- **Example:** `ssh -D 1080 user@remote-server`
- You configure your browser (e.g., Firefox) to use a "SOCKS5 Proxy" at `localhost:1080`.
- Now, every website you visit sees the IP of the **remote server**, not your own.
- This is a "poor man's VPN" that is very effective.

# Part V: Virtual Private Networks (VPNs)

# Tunnel vs. VPN

- **Tunnel:** Connects specific **ports**. (Granular).
- **VPN:** Connects entire **networks**. (Transparent).
- A VPN creates a virtual network interface (e.g., `tun0`) on your computer.
- All your traffic is automatically routed through the encrypted tunnel.

# Why Use a VPN?

- **Security:** Protects your data on public Wi-Fi.
- **Access:** Join the University network from home to access library resources.
- **Privacy:** Hides your activity from your ISP.
- **Global Access:** Access content restricted to certain regions.

# VPN Protocols: OpenVPN

- **Features:** Extremely flexible and robust.
- **Compatibility:** Works on almost every device.
- **Complexity:** Large codebase and difficult to configure manually.
- **Performance:** Can be slow due to high overhead.

# VPN Protocols: WireGuard

- **Features:** The modern standard for VPNs.
- **Speed:** Extremely fast with very low latency.
- **Simplicity:** Very small codebase (easier to find security bugs).
- **Modern:** Uses state-of-the-art cryptography.
- Built into the Linux kernel for maximum performance.

# Part VI: Advanced Infrastructure

## Load Balancing I

What happens when your service is too popular?

- Thousands of users try to connect to one server.
- The CPU hits 100%, and the memory fills up.
- The service crashes.

# Load Balancing II

A **Load Balancer** (like NGINX) is the entry point.

- It sits in front of a group of servers (a “cluster”).
- It receives all incoming connections.
- It decides which server is the least busy and forwards the traffic.
- If one server fails, the balancer stops sending traffic to it (High Availability).

# Intrusion Prevention: `fail2ban` I

The moment you put a server on the internet, it is attacked.

- Scripts (bots) will try to log in using common passwords.
- This is called a “Brute Force” attack.
- Even if they don’t get in, they consume your server’s resources.

# Intrusion Prevention: `fail2ban` II

`fail2ban` is the automated security guard.

- It monitors the logs of your services (SSH, Web, etc.).
- If an IP fails to log in 5 times in a row, `fail2ban` blocks it.
- It updates your **Firewall** (iptables/nftables) to drop all packets from that IP.
- This makes attacking your server much harder and more expensive for the hacker.

# Summary

# Summary

- **Measurement:** Use `iperf3` and `mtr` to quantify your network quality.
- **Synchronization:** Use `rsync` to move data efficiently and reliably.
- **Bridging:** Use SSH Tunnels to bypass firewalls and reach isolated services.
- **Expansion:** Use VPNs (WireGuard) to securely join remote networks.
- **Resilience:** Use Load Balancers and `fail2ban` to protect and scale your services.